

COMP 333

Project - Phase 1

Sofia Valiante	40191897
Nicolas Moscato-Barbeau	40210244
Terryl-Errol Foryoung	40127596
Gulnoor Kaur	40114998
Daniel Pace	40226833

November 3, 2024

Contents

1	Topic and Task Definition	3
2	Collected Datasets	3
3	Data Integration	4
3.1	Merging the Data Sets	4
3.2	Duplicate Removal	6
3.3	Functional Dependencies	6
4	Data Preparation	7
4.1	Null Values	7
4.2	Outlier Detection	7
4.3	Data Transformation	10
5	Tools and System	12
	References	i

1 Topic and Task Definition

The task is to estimate revenue generated by a movie based on its metadata.

The project consists of a regression task with multiple input values to determine an approximation of a single output value, i.e. the revenue.

We expect inputs like budget, genres, ratings, popularity, and release date to hold significant weight in the calculations. However, we might come to find that other factors also impact the revenue.

The main idea of this project is to estimate the revenue generated by a movie based on its metadata. The machine learning model is trained with the input data to predict the box office revenue. Some expected input values include attributes like budget, genres, release date, ratings, director, and actors that hold a significant weight in the predicting calculations. Moreover, this project consists of a regression task with multiple input values to determine an approximation of a single output value, i.e. the revenue. There might be some additional attributes that help to do the same.

2 Collected Datasets

The datasets utilized were found from two different sources. The first was taken from Kaggle, a data science competition platform [1]. It was downloaded as a CSV file, containing 1 table, 24 columns, and 3000 rows. The file measured 28.31 MB

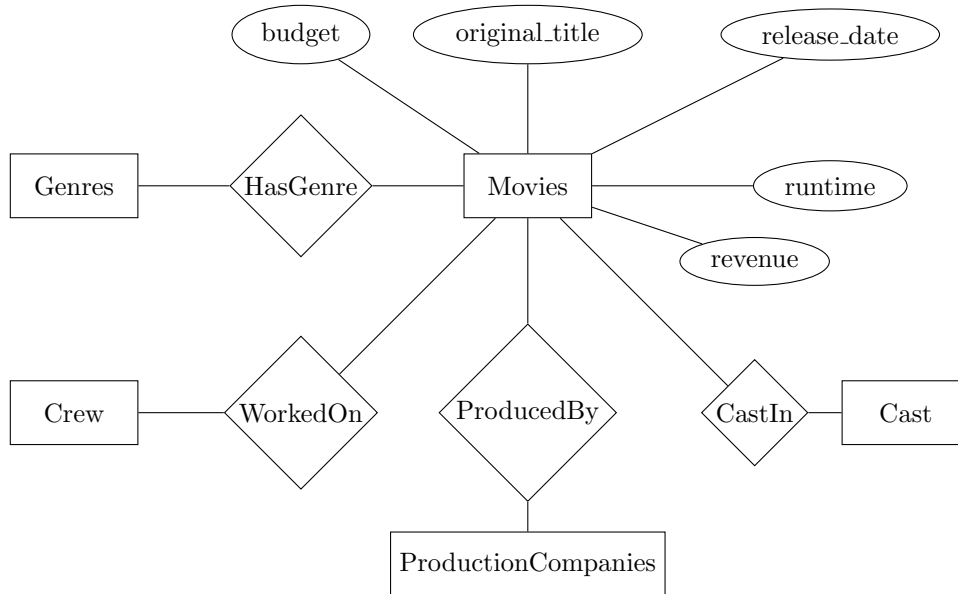


Figure 1: ERD for Kaggle Dataset

The second source was a repository on the Mendeley Data platform, a cloud-based repository [2]. The data was provided as a RAR file and extracted. It contained many different datasets from different sources. Two files CSV files were chosen to be used. These were chosen because of the usefulness of the data and because it was decided that we had sufficient rows.

The first of these files contained data extracted from IMDb. It contained 1 table with 28 columns and 5044 rows. The file measured 1.46 MB. The second file was sourced from Box Office Mojo, containing 1 table with 26 rows and 3244 rows. The file measured 1.54 MB.

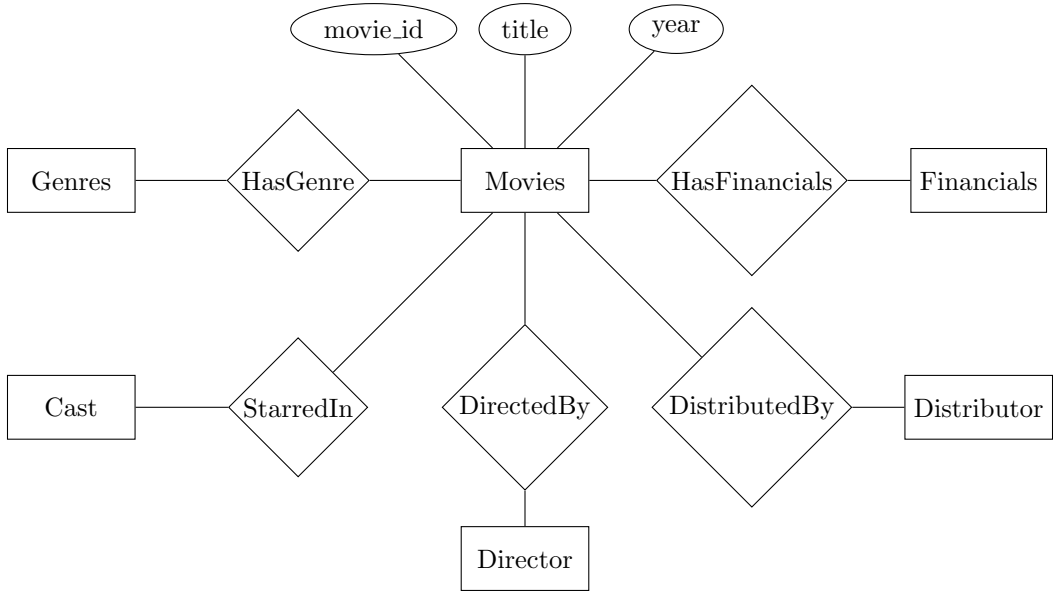


Figure 2: ERD for Mojo Box Office Dataset

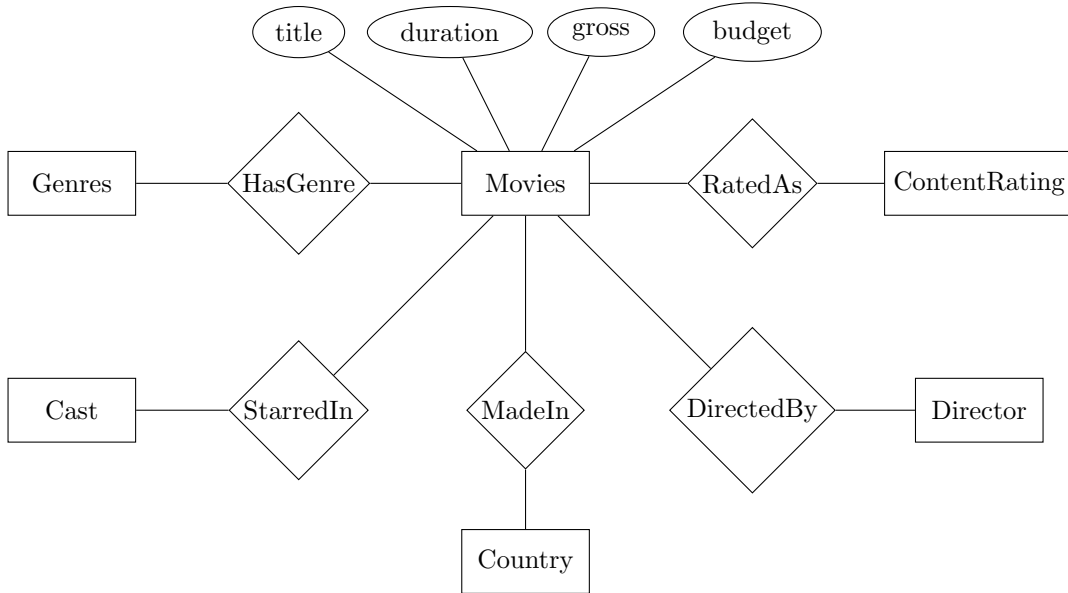


Figure 3: ERD for IMDb Dataset

3 Data Integration

3.1 Merging the Data Sets

Several format inconsistencies were discovered between the data sets that required further investigation. The dataset from Box Office Mojo was already in an acceptable format, so the other two datasets were modified to align closely with it.

Starting with the IMDb dataset, the **genre** column listed multiple genres for each movie, separated by a pipe (“|”) symbol, while the Mojo set had separate columns. To promote consistency, we used Python to split the genres into individual columns with the following code:

```

import pandas as pd

df = pd.read_csv('movie_metadata.csv')

genre_columns = df['genres'].str.split('|', expand=True)
genre_columns.columns = [f'genre_{i+1}' for i in range(genre_columns.shape[1])]

df = df.join(genre_columns)

df.drop('genres', axis=1, inplace=True)

df.to_csv('updated_dataset.csv', index=False)

```

Figure 4: Python code to split genres column.

The Kaggle dataset presented a similar, albeit tougher challenge, with columns like `cast`, `crew`, `genres`, and `production_companies` containing multiple JSON-formatted values in list form. Using Python (see Figure 2), we extracted the first four main actors into separate columns to match the Mojo dataset format. For genre, we retained only the first four genres per movie, and for production companies, we extracted only the first listed company. For the crew column, the director was identified and extracted using the `job` attribute in the JSON item.

```

import pandas as pd
import json

df = pd.read_csv('train.csv')

def get_genres(json_list_str):
    if isinstance(json_list_str, float):
        return [None]*4

    try:
        list= json.loads(json_list_str.replace("'", ''))
        genre_names = [genre['name'] for genre in list[:4]]
        return genre_names + [None]*(4-len(genre_names))
    except (json.JSONDecodeError, IndexError, KeyError):
        return [None] * 4

df[['genre_1', 'genre_2', 'genre_3', 'genre_4']] = pd.DataFrame(df['genres'].apply(
    get_genres).tolist(), index=df.index)

df.to_csv('updated_dataset.csv', index=False)

```

Figure 5: Python code to extract JSON items.

Another issue arose in the Kaggle dataset with release dates being formatted inconsistently, such as 2/20/15 (M/D/Y) for some, and others as 2008-06-04. The tools did not recognize the first format as a date. This was resolved by writing a script that parsed and reformatted these dates to match the other formats.

We then noticed that the runtimes were in different formats. The Kaggle and IMDb datasets reported runtime in minutes, while Mojo used an `Xhr Ymin` format (e.g., “1hr 30min”). Minutes seemed to be the better format, so we modified the Mojo dataset accordingly.

From the release date columns, the year was extracted for the tables that did not already have it, to make further data preparations simpler.

We then removed columns that the team deemed unnecessary, such as `rating`, `Facebook likes`, `aspect ratio`, `color`, and `imdb_link`. These columns were excluded for various reasons. Some, such as `imdb_link`, had no information helpful to the predictive model. Others, such as `rating`, would not be available when predicting the revenue of an unreleased/recently released film. The columns deemed to have predictive information were kept, including title, year, MPAA rating (parental rating), release date, distributor/production company, duration, director, the top four actors, the top three genres, budget, and revenue. The final column mapping between datasets is shown in Table 1.

Mojo	IMDb	Kaggle
title	movie_title	title
year	title_year	
mpaa	content_rating	
release_date		release_date
run_time	duration	runtime
distributor		production_companies
director	director_name	crew
main_actor_1	actor_1_name	cast (JSON List)
main_actor_2	actor_2_name	cast (JSON List)
main_actor_3	actor_3_name	cast (JSON List)
main_actor_4		cast (JSON List)
budget	budget	budget
worldwide	gross	revenue
genre_1	genres (List)	genres (JSON List)
genre_2	genres (List)	genres (JSON List)
genre_3	genres (List)	genres (JSON List)
genre_4	genres (List)	genres (JSON List)

Table 1: Column Mapping Between Datasets

To facilitate the merging process, the corresponding columns were renamed to a uniform schema. Using Pandas, the CSV files were loaded into data frames, merged, and then output to a new CSV file.

3.2 Duplicate Removal

Next, we used Excel’s data tools to check for and remove duplicates based on the title and year. Some duplicates, however, remained due to a difference in international release date, particularly the year, resulting in multiple records for the same film with a different year. We used a Python script to eliminate these remaining duplicates if the difference in the year was 1.

```
import pandas as pd

df = pd.read_csv('merged.csv')

df = df.sort_values(by=['title', 'year']).reset_index(drop=True)

df['Near_Duplicate'] = (df['title'] == df['title'].shift()) & (df['year'] - df['year'].
    shift() == 1)

df_filtered = df[~df['Near_Duplicate']].drop(columns=['Near_Duplicate'])

df.to_csv('merge_noDupes.csv', index=False)
```

Figure 6: Python code find films with same title release 1 year form each other.

3.3 Functional Dependencies

The main functional dependency identified during the data integration process was:

(Title, year) $\rightarrow$ mpaa, release_date, distributor, director, main_actor_1, main_actor_2,
main_actor_3, main_actor_4, budget, revenue, genre_1, genre_2, genre_3, runtime

Or more simply, (Title, year) is the candidate key. Overall this is a good outcome for our dataset. We are not planning to use the title to train the model and it is preferable when training a model to not have any unique identifiers. This way we have a candidate key to identify relations in our data but can still train our model effectively.

4 Data Preparation

4.1 Null Values

- **Method:** Since we have the tool to get data we will try to fill in the missing data with that. To handle null values in our data we decided to remove the relations with null values. This is the simplest way to handle this case. While it may not be the best way to handle these cases, we feel that we collected enough relations and do not need these. Especially since other methods like filling in the values might introduce some other errors. For example, retrieving names could introduce more duplicates.
- **Examples:** Attributes in which the entire record was removed due to a null value that couldn't be supplemented from another dataset were directors, revenue, missing cast, and distributor.
 1. The director is missing

1982		1982-12-07	Paramount Pictures
------	--	------------	--------------------

Figure 7: Missing Director

2. The revenue is missing

revenue	genre_1	genre_2	genre_3	genre_4	run_time	Near_Duplicate
	Drama	Horror	Mystery	Thriller	101	FALSE

Figure 8: Missing Revenue

3. The entire cast is missing

2004		2004-11-03	Columbia Pictures				
------	--	------------	-------------------	--	--	--	--

Figure 9: Missing Cast

4. The distributor is missing

2015	Not Rated			Tara Subkoff	Timothy Hutton	Balthazar Getty	Lydia Hearst
------	-----------	--	--	--------------	----------------	-----------------	--------------

Figure 10: Missing Distributor

4.2 Outlier Detection

- **Method:** There was more than one method used to determine outliers. The simplest of which was by observation and scanning the file. We noticed that there existed some data where the value did not match the context of the attribute, such as if the budget of a movie was zero dollars.

Another method for outlier detection was to graph multiple data points and observe if any outliers existed. As such, outlier detection could not be performed for directors, distributors, and main actors as there is no question if there was a miscalculation or improper measure. The attributes are very much real-world objects.

Z-score normalization and outlier analysis was used for monetary values such as budget and revenue. For a normal distribution, it is found that 99.7% of the data points lie between ± 3 standard deviation. Thus, any z-score with a value greater than 3 was removed from the dataset.

Another outlier detection method was to confirm if the labels for the release date exceeded the value 366. If there were, these values should be identified as an outlier since there are

only 365 days in a year and 366 if it's a leap year. As such, the method was performed on the dataset but no records were found that violated the condition.

Another outlier detection method was to remove movies that had a year less than 1900, as including older films in our dataset did not seem realistic. However, no movies were found to have violated the condition.

- **Examples:** Some examples of cases where outliers were detected include
 1. The extreme values in the budget were examined to determine the valid data points and the error values. The values are zero which is not realistic and were few and far between, therefore they were removed completely.

budget	
1500000	
0	
7500000	
15000000	
15000000	
12000000	
0	

Figure 11: Budget Outlier Example

2. The attribute runtime also had values equal to zero which was not realistic for the length of a movie and was identified as noisy data, possibly due to a measuring error. In the example below, the runtime is zero.

6843500	10703234	Comedy				0	FALSE
---------	----------	--------	--	--	--	---	-------

Figure 12: Runtime Outlier Example

3. There were cases where revenue was zero. Given that revenue is the most important attribute for our dataset as we wish to predict it, we determined that only precise and qualified revenues would be sufficient, therefore any revenue of value zero was removed.

2008			Frédéric Forestier	Alain Delon	Santiago Segura	Vanessa Hessler	78000000	0	Adventure	Comedy	Family	Fantasy
------	--	--	--------------------	-------------	-----------------	-----------------	----------	---	-----------	--------	--------	---------

Figure 13: Revenue Outlier Example

- These values for revenue were observed as well and removed.

2006	2006-05-14	Кинокомпания «Лупарки»						750000	3	Crime	Comedy
2001	2001-09-29	ABS-CBN Film Productions						344	4	Comedy	Romance

Figure 14: Remove outlier

- Graphing to determine outliers. Here it can be noted distributors between 150 exclusive and 200 inclusive have no release dates. This must be some anomalous data. The data could be missing or it could be that the release_date is in an incorrect format. Adjustments were made.

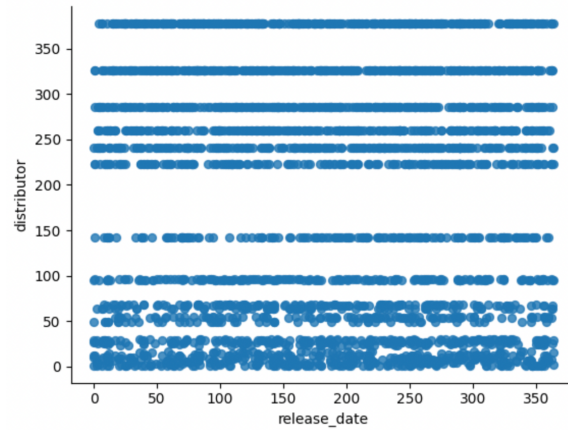


Figure 15: Release Date by Distributor

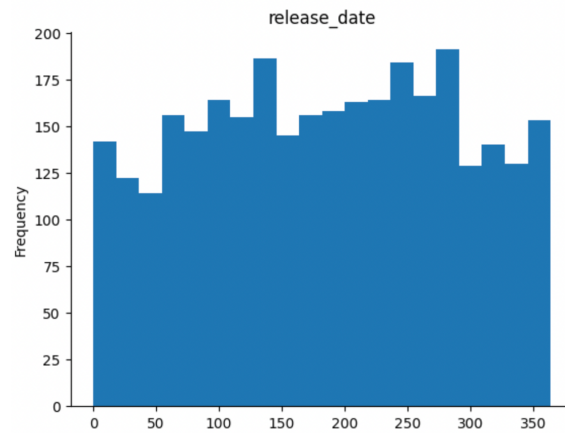


Figure 16: Release Date Histogram

- Z-score outlier detection. Note some values higher than 3, like 3.810094, were removed.

2004.0	2	49	3	2	1	9	1	1.0	1.045252	0.664990
1998.0	4	12	1	10	1	1	1	1.0	1.045112	0.666069
2004.0	3	94	6	1	1	1	1	1.0	1.045111	0.663512
2007.0	4	299	241	1	2	2	1	1.0	1.044951	0.254377
1990.0	4	53	1	7	1	1	1	1.0	1.044484	0.657086
...	...	...	...	...	...	...	...	...	...	...
2019.0	3	206	223	6	1	1	1	1.0	3.810094	3.481955
2018.0	3	272	223	13	1	6	2	1.0	3.898073	0.878599
2007.0	3	260	223	10	25	2	4	2.0	5.485749	3.909801
2017.0	3	146	223	3	2	2	1	1.0	4.792238	4.701094
2019.0	3	276	223	4	12	6	8	2.0	5.240222	10.138672

Figure 17: Z-scores for attributes

2004.0	2	49	3	2	1	9	1	1.0	1.045252	0.664990
1998.0	4	12	1	10	1	1	1	1.0	1.045112	0.666069
2004.0	3	94	6	1	1	1	1	1.0	1.045111	0.663512
2007.0	4	299	241	1	2	2	1	1.0	1.044951	0.254377
1990.0	4	53	1	7	1	1	1	1.0	1.044484	0.657086
...	...	...	...	...	...	...	...	...	...	...
2014.0	3	248	286	9	8	4	4	1.0	2.768130	2.445126
2017.0	3	288	223	4	6	5	2	2.0	2.637697	2.812837
2018.0	3	142	378	5	3	3	4	1.0	2.549886	1.908499
2019.0	3	54	286	1	7	3	3	3.0	2.485902	0.308565
2017.0	3	246	241	13	22	9	4	1.0	2.950750	1.772283

Figure 18: Outliers Removed

4.3 Data Transformation

- **Methods:** Some data transformation occurred during data integration as the datasets all had different formats for certain attributes, such as release_date and run_time. Thus, the format of release_dates and run_time was standardized to YYYY-MM-DD and minutes respectively.

Genres were converted from strings to numeric format performing label encoding. Furthermore, there existed genres that were missing for some records in the dataset. Given that there were multiple attributes for the genre and therefore already decently sufficient data on the genres of the movie, missing genres were replaced with duplicates of the movie's genre_1.

The monetary attributes, namely revenue and budget, were adjusted for inflation to have all values be representative of the current time period, and as such gain an understanding of the monetary returns of all movies.

Mpaa and release_date were label encoded as their values were unique and do not hold any significance from their context as a director or distribution studio would.

After the revenue and budget were adjusted for inflation, to transform the attributes into a format more suitable for a machine learning model, they were transformed using Z-score

normalization. The absolute values of both revenue and budget were taken.

The attributes distributor, director, main_actor_1, main_actor_2, main_actor_3, and main_actor_4 were originally strings that needed to be transformed into a numerical format while preserving their significance. Distributors, directors, and actors play a crucial role in influencing a new movie's potential revenue, as their reputations can draw audiences based solely on name recognition. For instance, a well-known actor or director can significantly boost ticket sales, while larger film production companies tend to invest more resources into a project, enhancing its quality and appeal. To maintain the contextual importance of these values during encoding, we performed ordinal encoding on the values of the attributes. This method involved replacing each attribute's value with the frequency of its occurrence in the dataset, assigning greater significance to more frequently appearing names.

In the special cases where the budget was zero, the entire record was removed. However, for the cases where the budget was missing or null, we supplemented the value by taking the average of both the median of distributor rating and the median of main_actor rating added together.

- **Examples:**

1. Attributes genre_1, genre_2, and genre_3 were changed in numeric format by label encoding.

				genre_1	genre_2	genre_3	genre_4
				7	12	16	19
				17	0	7	20
				7	12	16	16
				11	5	23	5
genre_1	genre_2	genre_3	genre_4	0	1	7	7
Drama	Horror	Mystery	Thriller	...	...	...	...
Thriller	Action	Drama	War	4	8	9	15
Comedy	Drama	Romance	NaN	0	1	8	16
Thriller	Science Fiction	Drama	NaN	2	1	8	3
				0	1	5	19
Drama	Horror	Mystery	Sci-Fi	0	1	19	19

Figure 19: Label Encoding of Genre

2. Below is an example of the budget before and after adjusting for inflation.

budget		revenue		budget		revenue	
1500000.0		NaN		1.693513e+07	8.117258e+07		
0.0		1625847.0		1.321479e+08	3.395374e+08		
				5.645044e+07	7.836604e+07		
7500000.0		60722734.0					
				9.077398e+07	1.388919e+07		
15000000.0		108286421.0					
				9.180237e+07	8.623300e+07		
				...			
15000000.0		71897215.0					
				2.980787e+07	1.293259e+08		

Figure 20: Inflation Adjustment

3. In order to deal with multiple null values in the columns related to genre, the column genre_4 was dropped, and the missing values in columns genre_2 and genre_3 were replaced with values from genre_1. In the example, the label 7 is repeated twice.

Figure 21: Null Genre Resolution

4. Label encoding attributes MPAA and release_date.

3128	Independence Day: Resurgence	2016.0	3	258	Twentieth Century Fox	Roland Emmerich
3132	Alita: Battle Angel	2019.0	3	63	Twentieth Century Fox	Robert Rodriguez

Figure 22: Label Encoding of MPAA and Release Date

5. Ordinal encoding of distributor, director, and all main actors as they're in string format originally.

<pre>distributor Warner Bros. 378 Universal Pictures 326 Twentieth Century Fox 286 Sony Pictures Releasing 260 Paramount Pictures 241 ... Slugger Pictures 1 Indican Pictures 1 Cowboy Pictures 1 Variance Films 1 Clarius Entertainment 1 Name: count, Length: 162, dtype: int64</pre>							
7	Paranormal Activity	2007.0	4	299	Paramount Pictures		
7	Paranormal Activity	2007.0	4	299	241	1	2

Figure 23: Ordinal Encoding of String Attributes

5 Tools and System

- **Google Colab:** Used for collaborative coding and MLM.
- **Python:** Coding language used for all data integration and preparation
- **Power BI:** Used for data visualization.
- **Excel:** Used for viewing datasets and preliminary data cleaning.
- **ChatGPT:** Used for basic questions and debugging code.
- **Overleaf:** Used for formatting this report and creating ERD diagrams.

References

- [1] A. Howard, “Tmdb box office prediction.” <https://kaggle.com/competitions/tmdb-box-office-prediction>, 2019. Kaggle.
- [2] C. Madongo, “Movie box office revenue prediction.” <https://doi.org/10.17632/xv9wtc9gdk.2>, 2020.